

Lecture 12: LLMs & the Limits of Data

LSE ME314: Introduction to Data Science and Machine Learning (<https://github.com/me314-lse>)

2026-07-30

Ryan Hübert

Where we left off yesterday

Yesterday ended with the document-feature matrix: text as counts

→ Useful for many tasks: classification, sentiment, etc.

But it has two problems:

1. It represents text inefficiently
 - Every distinct word is a column: huge, sparse, unbounded
 - All columns are unrelated: “dog” is no more similar to “puppy” than to “refrigerator”
2. It is static at the corpus level
 - One meaning per word, wherever it appears: “bank” is one column, money or river

Both are forms of information loss, the same problem as all week

Today: how modern language models fix both problems

Why fix these problems?

Better representations of text are useful in two ways:

1. As better inputs for yesterday's tasks
 - Classification, extraction, measurement
 - This route leads to encoder models (like BERT)
2. Once representations are good enough, models *generate* text
 - Predict the next word, over and over
 - This route leads to generative LLMs (GPT, Claude, Gemini)

We will see both today

Getting there: better tokens, then better vectors, then context

What is a large language model?

What is a large language model?

A **large language model** is a large (estimated) model of language.

Language model:

- Language is basically a sequence of utterances
- Can represent these as sequence of tokens $x_1, x_2, \dots, x_T$
- A language model is a statistical model that predicts tokens from other tokens in the sequence

Large:

- Billions (possibly trillions) of model parameters
- Can work with very large amounts of text: hundreds of thousands, or even millions, of words

The *model* in an LLM

There are two big sets of issues:

1. **Training**: estimating the model

- Happens once, and model (“**weights**”) is estimated

2. **Inference**: using the model for tasks

- In ML context: “predicting” on new data
- Some LLMs used for prediction (encoder models)
- Some used to generate text (decoder models, “generative AI”)

We’re going to talk about inference *then* training

- Inference will be very useful for you
- Training might be useful, but it is good background knowledge

Some early caveats and notes

- LLMs are relatively new, we're still figuring out how to use them
- Technical advances in LLMs are occurring rapidly
 - Good idea to keep up-to-date: read news coverage, follow new releases, try to understand advances
- There are *many* LLMs out there
 - Here: we're focused on high-level ideas + some examples
 - Details will vary from model to model
- Need to distinguish between companies, models and consumer products
 - E.g.: Anthropic versus Fable 5 versus Claude

Mind your language

We describe LLMs with human words

- It “thinks”, “reasons”, “understands”, “lies”, “apologises”
- Notice the quotes: LLMs aren’t human!

This is **anthropomorphisation**: importing a human mental model

- AI companies lean into it (e.g., *Anthropic*)
- Hard to avoid since natural language is the interface

Why it matters: a human mental model trains you to expect the wrong things

- A human who apologises understood the mistake, but the model superficially updated on your pushback
- A human who sounds confident usually has reasons, but the model *a/ways* sounds confident
- Treat the human words as metaphors, not descriptions

The LLM is just the model

You know this distinction from ML: the model versus the software you use it through

- The LLM is the model: estimated weights that predict tokens from tokens
- That is *all* the model does

Everyday speech blurs this: “LLM” often means a whole software product

- A **chatbot** wraps a model in a conversational interface
 - E.g.: the ChatGPT, Claude and Gemini apps
- A **harness** wraps a model in software that carries out actions the model requests
 - An LLM working through a harness is called an **agent**
 - E.g.: Claude Code, Codex, Gemini CLI

When a product “remembers” you, “searches the web” or “runs code”, that is the software, not the model

How LLMs quantify texts

How LLMs tokenise: byte pair encoding

As before: LLMs need to start with quantified texts.

Also as before: LLMs start by tokenising strings.

LLMs use **modified byte pair encoding (BPE)** tokenisers

- Tokens are bundles of commonly occurring consecutive substrings of Unicode characters
- A lookup table (like a character set) where every distinct token has an ID number
- Keep in mind: different LLMs may use different tokenisers

E.g., OpenAI's recent models use the o200k_base tokeniser

- It has about 200,000 unique tokens

How LLMs tokenise: byte pair encoding

OpenAI's tokeniser is available in R via the `{rtiktoken}` package

```
library("rtiktoken")  
get_tokens(text = "London School of Economics",  
           model = "o200k_base")
```

```
[1] 51162 6586 328 56779
```

```
get_token_count(text = "London School of Economics",  
                model = "o200k_base")
```

```
[1] 4
```

If you have a token ID, you can decode:

```
decode_tokens(as.integer(51162), model = "o200k_base")
```

```
[1] "London"
```

How LLMs tokenise: byte pair encoding

What makes BPE tokenisers different from “traditional” tokenisers:

```
get_tokens(text = "london school of economics", model = "o200k_base")
```

```
[1] 75 8552 3474 328 48229
```

With this BPE tokeniser, “london” is two tokens!

- Why? Short answer: “London” is a more common set of co-occurring bytes than “london”
- BPE tokenisers yield more efficient representations

```
sapply(as.integer(c(75,8552,3474,328,48229)),  
       decode_tokens, model = "o200k_base")
```

```
[1] "l"      "ndon"    " school" " of"     " economics"
```

Play around here: <https://platform.openai.com/tokenizer>

Tokens to vector representations

Tokenising breaks up text into parts, but LLMs don't use tokens directly

Tokens are converted into vector representations (VRs)

- Each token is mapped to a specific vector in a smaller dimensional vector space
- Similar in spirit to PCA

Why is this better than working with tokens directly?

- Preview: vectors are denser, and they can capture *relationships* between words
- The next few slides develop this idea, then return to the “why” properly

Tokens as dimensions in a vector space

Bag of words: each word has a distinct, unique meaning

→ In math terms: we treat words as **one-hot encodings**

Concrete example: consider a vocabulary of three words:
`c("cat", "dog", "rat")`

Can represent each word in **vector format**:

→ The word cat is $(1,0,0)$

→ The word dog is $(0,1,0)$

→ The word rat is $(0,0,1)$

(Should be obvious why this is called “one-hot encoding”)

These are *orthogonal*: zero similarity between the words

These are dimensions of a vector space

Word embeddings

But what if words are actually representations of a smaller group of concepts, where each word is a *mix* of concepts?

E.g. what if these words can be represented in a two dimensional **embedding** (e.g. two distinct “concepts”)

Then, you might have

- The word cat represented by (0.39,0.61)
- The word dog represented by (0.23,0.77)
- The word rat represented by (0.82,0.18)

Word embeddings have to be *estimated* from a corpus

- Key idea: words appearing in *similar contexts* get *similar vectors*
- Classic methods: **word2vec**, **GloVe** (downloadable)

Estimating embeddings: a tiny example

We can estimate embeddings ourselves using PCA: count which words appear *near* which, then reduce dimensions

```
docs <- c("the dog chased the cat", "my dog is a loyal pet",  
          "the cat is a lazy pet", "a cat and a dog make good pets",  
          "deposit money in the bank", "the bank raised interest rates",  
          "she withdrew money from the bank",  
          "invest money in the stock market")  
toks <- tokens(docs, remove_punct = TRUE)  
toks[1:3] # show first three
```

Tokens consisting of 3 documents.

text1 :

```
[1] "the"      "dog"      "chased" "the"      "cat"
```

text2 :

```
[1] "my"      "dog"      "is"      "a"      "loyal" "pet"
```

text3 :

```
[1] "the"      "cat"      "is"      "a"      "lazy" "pet"
```

Estimating embeddings: a tiny example

```
X <- fcm(toks, context = "window", window = 3) # co-occurrence
iw <- c("the", "dog", "cat", "money", "bank")
X[row.names(X) %in% iw, colnames(X) %in% iw] # show 5 words only
```

Feature co-occurrence matrix of: 5 by 5 features.

	features				
features	the	dog	cat	money	bank
the	2	2	2	3	3
dog	0	0	2	0	0
cat	0	0	0	0	0
money	0	0	0	0	2
bank	0	0	0	0	0

Estimating embeddings: a tiny example

```
X <- as.matrix(X) + t(as.matrix(X)) # make a symmetric matrix
X[row.names(X) %in% iw, colnames(X) %in% iw] # show 5 words only
```

	features				
features	the	dog	cat	money	bank
the	4	2	2	3	3
dog	2	0	2	0	0
cat	2	2	0	0	0
money	3	0	0	0	2
bank	3	0	0	2	0

Each word's row of the co-occurrence matrix is its **context vector**: which words it appears near, and how often.

Estimating embeddings: a tiny example

```
emb <- prcomp(X) # PCA on the co-occurrence matrix
emb$x[row.names(emb$x) %in% iw,1:12]
```

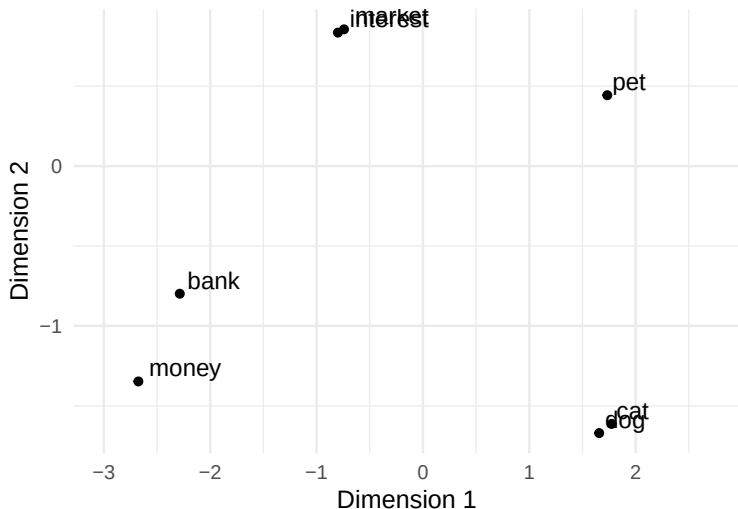
features	PC1	PC2	PC3	PC4	PC5	PC6
the	-2.892000	-5.3316207	1.24418124	0.5162346	0.15275110	-0.8017542
dog	1.657206	-1.6703213	-1.58280338	-0.3575886	0.98228531	-0.2487849
cat	1.776029	-1.6127397	0.89316374	0.3194582	0.52251822	2.0446082
money	-2.675874	-1.3474197	-0.01327316	-2.3074555	-0.26672653	0.6033245
bank	-2.286200	-0.7985788	-1.28716414	1.2442770	-0.06870046	0.7343113

features	PC7	PC8	PC9	PC10	PC11	PC12
the	0.16251601	-0.06155100	-0.1430913	0.3566172	-0.1729570	0.05577974
dog	-0.02839386	-1.35795409	-0.1140250	-0.1705859	-0.3363425	0.16764744
cat	0.18632051	0.16963982	0.2288980	-0.3266659	-0.1448522	-0.05084265
money	-0.60025339	0.06024335	0.2847971	0.3082000	0.2672553	0.08817187
bank	0.83304710	0.18392816	0.3505633	0.4339756	0.4145016	0.15124609

Each word's embedding: its first few **principal components**.

- Exactly the dimension reduction idea from Lecture 9
- word2vec and GloVe do (roughly) this at scale, more cleverly

Estimating embeddings: a tiny example



Dimension 1 separates *pets* from *finance*. Nobody told the machine those concepts exist: it found them in the co-occurrence patterns.

Measuring similarity between word embeddings

Similarity of context vectors (before embedding):

```
cat.cvec <- X[row.names(X) == "cat", ]  
dog.cvec <- X[row.names(X) == "dog", ]  
bank.cvec <- X[row.names(X) == "bank", ]  
  
cosine <- function(a, b) sum(a * b) / sqrt(sum(a^2) * sum(b^2))  
  
cosine(cat.cvec, dog.cvec)
```

```
[1] 0.634335
```

```
cosine(cat.cvec, bank.cvec)
```

```
[1] 0.3086067
```

Measuring similarity between word embeddings

Similarity of embedded words:

```
cat.emb <- emb$x[row.names(emb$x) == "cat", 1:2]  
dog.emb <- emb$x[row.names(emb$x) == "dog", 1:2]  
bank.emb <- emb$x[row.names(emb$x) == "bank", 1:2]  
  
cosine(cat.emb, dog.emb)
```

```
[1] 0.9986436
```

```
cosine(cat.emb, bank.emb)
```

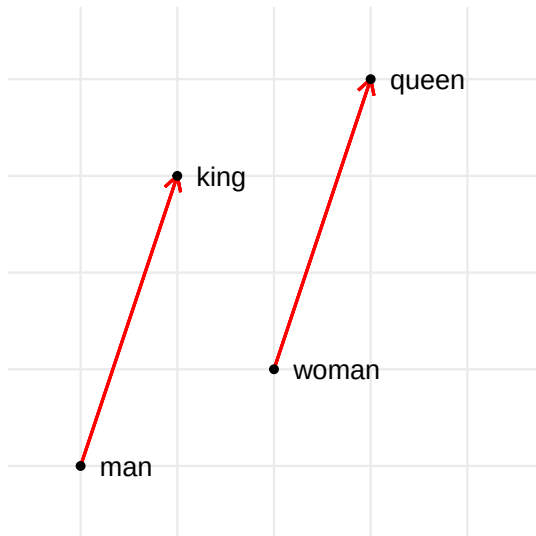
```
[1] -0.4772226
```

Note: to estimate embeddings (vector representations of tokens), analysts usually don't use PCA

- You know PCA, and you *can* estimate embeddings this way
- Technical details are beyond our scope

Embeddings capture analogies

Analogy: king - man + woman = queen



Why embeddings?

Use embeddings, not raw words, as features for text analysis. Why?

1. **Similarity is built in:** “dog”, “puppy”, “canine” get similar vectors
 - One-hot: “dog” and “puppy” are as unrelated as “dog” and “refrigerator”
2. **Denser and smaller:** far fewer dimensions than a sparse DFM
 - Models generalise better from denser features
3. **Geometry captures analogies:** $\text{king} - \text{man} + \text{woman} \approx \text{queen}$

LLMs are built on the same idea: step one is to embed every token

This solves problem 1, but problem 2 remains: each word gets *one fixed vector* wherever it appears

VRs in LLMs

An LLM's first step after tokenising: map every token to a vector

- A lookup table: each token ID gets its own vector
- Same idea as the word embeddings you just saw, but per BPE *token*, not per word
- Learned jointly with the rest of the model during training, not estimated from co-occurrence counts
- Some models expose their VRs (Llama), some don't (GPT-5)

Also called *token embeddings*; we say VRs to avoid a name clash

But the lookup is still *static*: one vector per token, whatever the sentence

- Problem 2 stands: “bank” the river and “bank” the money share a vector
- Fixing that is the transformer's job: next section

Transformers: VRs in context

Transformer architecture: big picture

Where we are: every token has a vector, but the *same* vector in every sentence

The transformer exists to fix exactly this

- It takes the static VRs and *rewrites* them, step by step
- Each rewrite blends in information from the other tokens in the prompt
- Out the other end: a **bespoke** vector for every token, reflecting the sentence it actually sits in

The magic is the iteration: not one lookup, but vectors *improved* many times over

Next: how (“attention”), and why this was a breakthrough

How does the model “understand” language?

In language, meaning of a word depends on other words around it

- “bank” means money in one sentence, a riverbank in another
- Pronouns, references, topics, tone: all spread across long spans of text

We’ll loosely call this the “context” of the word

Core challenge for a statistical model of language: determine “context” from a sequence of given tokens (from prompt)

Previous algorithms (like RNNs) processed tokens in order, but:

- Created computational bottlenecks
- Made long-range context hard to capture
- Slow to train these models; generated text not as good

How does the model “understand” language?

Huge break-through in 2017: **transformers**

We won't get into the details, but:

- Transformers use an **attention mechanism** where context is determined from all tokens in parallel (not in sequence)
- Vastly sped up training, and made inference much better

Static embeddings look words up; transformers let the sentence edit the lookup

Forward pass on the prompt → context

Forward pass on the prompt: VRs from the original prompt are “passed” through the transformer repeatedly

- Each pass is called a **transformer block** or a **layer**
- There are a fixed number of layers
- In each layer, all the VRs are updated to new VRs

After final block, the resulting (modified) VRs are called the **hidden states** for each of the original tokens

- These are mathematical representations of the “context” for each of the original prompt tokens
- Sometimes called **contextual embeddings** since now each token’s vector reflect the “context” around it

Two kinds of transformer models

Hidden states solve problem 2: a vector per occurrence, not per word

What do you do with them? The two routes from the start of today:

1. Use the embeddings directly: **encoder models** (e.g., BERT)
 - Classification, extraction, representation
 - Many “text as data” workhorses live here
2. Add a language modelling head: **generative LLMs** (e.g., GPT, Claude)
 - Use context to predict the next token, over and over
 - This is the “talking” branch: chatbots, agents

Most of this lecture is about branch 2, but first, a look at branch 1

Contextual embeddings in R

The `{text}` package gives access to encoder models like BERT

→ Downloads the model from Hugging Face

```
# install.packages("text"); text::textrpp_install() # one-time setup
library("text")
textrpp_initialize()
sents <- c("I deposited cash at the bank.",
           "The bank approved my loan.",
           "We sat on the bank of the river.")

emb <- textEmbed(sents, model = "bert-base-uncased",
                 keep_token_embeddings = TRUE)
```

Contextual embeddings in R

BERT returns a vector for *every token in every sentence*

→ Three sentences, three different vectors for “bank”

```
text1.emb <- emb$tokens$texts[[1]]  
print(text1.emb[text1.emb$tokens=="bank",])
```

```
# A tibble: 1 × 770
```

```
  tokens      id Dim1   Dim2   Dim3   Dim4   Dim5   Dim6   Dim7   Dim8   Dim9   Dim10  
  <chr>   <int> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  
1 bank         1 0.685 -0.766 -0.162 0.298 0.885 0.0951 -0.598 0.911 -0.167 -0.23  
# 755 more variables: Dim14 <dbl>, Dim15 <dbl>, Dim16 <dbl>, Dim17 <dbl>, Dim18  
#   Dim20 <dbl>, Dim21 <dbl>, Dim22 <dbl>, Dim23 <dbl>, Dim24 <dbl>, Dim25 <dbl>  
#   Dim27 <dbl>, Dim28 <dbl>, Dim29 <dbl>, Dim30 <dbl>, Dim31 <dbl>, Dim32 <dbl>  
#   Dim34 <dbl>, Dim35 <dbl>, Dim36 <dbl>, Dim37 <dbl>, Dim38 <dbl>, Dim39 <dbl>  
#   Dim41 <dbl>, Dim42 <dbl>, Dim43 <dbl>, Dim44 <dbl>, Dim45 <dbl>, Dim46 <dbl>  
#   Dim48 <dbl>, Dim49 <dbl>, Dim50 <dbl>, Dim51 <dbl>, Dim52 <dbl>, Dim53 <dbl>  
#   Dim55 <dbl>, Dim56 <dbl>, Dim57 <dbl>, Dim58 <dbl>, Dim59 <dbl>, Dim60 <dbl>  
# Use `colnames()` to see all variable names
```

Contextual embeddings in R

How similar are the three “bank” vectors?

```
bank <- lapply(emb$tokens$texts, function(s)
  as.numeric(s[s$tokens == "bank", -1]))

cosine(bank[[1]], bank[[2]]) # deposit cash vs approve loan
cosine(bank[[1]], bank[[3]]) # deposit cash vs river
```

```
[1] 0.85
```

```
[1] 0.58
```

The money banks are much more similar to each other than to the river bank

- The word embeddings from earlier: one vector, all three identical
- The transformer has separated the meanings, using context

Inference: generating text

Inference: where generation begins

From here on, we follow common usage: “LLM” means the *generative* kind

The pipeline so far is shared by both branches:

- Prompt → tokens → static VRs → transformer → hidden states
- This is the **forward pass**: the model “reads” the prompt
- Nothing has been generated yet

Generative models add one component: the **language modelling (LM) head**

- Input: the hidden states (the context)
- Output: a probability distribution over the entire vocabulary
- In words: “given everything so far, what token comes next?”

Generating a reply is just asking that question over and over

Context → text generation

After forward pass on the prompt, model has determined “context,” but no text has been generated yet

This happens in the LM head:

- Use context to create a prob. distribution over all tokens
- Sample first token of response
- Pass this new token through the transformer
- Use (slightly modified) context to create a new prob. distribution over vocabulary
- Sample second token of response
- And so on
- Stop when hard limit is reached or next token is a “stop” token

When done, you have a reply and input and output tokens are contextualised (can continue the conversation)

The “temperature” of the text generation

In the LM head, tokens are sampled to generate text, but we didn't say *how* they were sampled

Temperature controls how the sampling occurs

- ➔ Lowest temperature: always pick the highest-probability token (deterministic)
- ➔ Higher temperatures: spread the probabilities out (more randomness and variation)

Note: for some models (e.g., recent GPT models), you cannot get deterministic outputs even with lowest temperature

Context windows

There is a limit to how many tokens a model can contextualise

- A **context window** specifies the maximum number of tokens a model can “attend to” in a session/conversation
- Includes all tokens input (e.g., in prompts) and all tokens output by the model (e.g., in responses)

When context window is reached: the *software* must drop or summarise older tokens (the model cannot “forget”, it just sees less)

Different models have different context windows:

- GPT-5.4: 1.05M context window
- Claude Opus 4.6: 200K context window (1M for some tiers)
- Gemini 3.1 Pro: 1.05M context window

Performance can degrade well before context window is reached

The model is stateless

An LLM has no memory

- Each response is generated fresh from the tokens it is given
- Everything it “knows” about your conversation is in the context window
- “Memory” features in chatbots: software re-inserting text into context

Implication: **context is your only lever**

Every technique for getting better outputs is a way of getting better text into the context:

- Prompts and instructions (later today)
- Examples (few-shot prompting)
- Retrieved documents (RAG)
- Tool results (agents)

Inference in R

You (probably) know how to do inference in the ChatGPT, Claude, Gemini and/or Copilot web apps

But you can do inference more programmatically in R:

- You can prompt some models locally
- You can prompt other models by calling web APIs

We'll use local prompting here: it's free! (APIs work similarly, but need an account and payment)

Two practical problems with local prompting:

1. Many models too computationally hungry for personal devices
2. Even for small models, infrastructure set up is too complex for your average user

Local inference in R

There is a path forward!

- Some “small” models, like versions of Meta’s Llama 3.2
- Ollama is a **local LLM runtime**: software that helps you run LLMs on a computer with almost no setup
 - Install from <https://ollama.com/>
- Ollama is software, not a model: it provides the platform, but you still need to download models

In R, we’ll use `{ollamar}` to interact with models via Ollama

```
# install.packages("ollamar")  
library("ollamar")
```

Local inference in R

```
test_connection() # Launch Ollama app if not status 200
```

Ollama local server running

```
list_models() # Which models have I already downloaded?
```

	name	size	parameter_size	quantization_level
1	llama3.2:3b	2 GB	3.2B	Q4_K_M

```
my.model <- "llama3.2:3b"  
if (!model_avail(my.model)) {  
  pull(my.model)  
}
```

Local inference in R

```
response <- generate(model = my.model,  
                      prompt = "When was LSE founded?",  
                      output = "text")  
cat(response)
```

The London School of Economics (LSE) was founded in 1895.

You can control the temperature:

```
cat(generate(model = my.model,  
             prompt = "Give me a sentence with 7 words.",  
             output = "text", temperature = 0))
```

The sun was shining brightly in California.

```
cat(generate(model = my.model,  
             prompt = "Give me a sentence with 7 words.",  
             output = "text", temperature = 0))
```

The sun was shining brightly in California.

Local inference in R

```
cat(generate(model = my.model,  
             prompt = "Give me a sentence with 7 words.",  
             output = "text", temperature = 2.8))
```

The dog is running around the house.

```
cat(generate(model = my.model,  
             prompt = "Give me a sentence with 7 words.",  
             output = "text", temperature = 2.8))
```

The big brown bear climbed the mountain.

Training

Training

So far, just talked about inference/prediction

However, each model (like GPT-5) has to be trained

Typically happens in two parts:

1. **Pretraining**: use large corpus of texts to learn general patterns, grammar, facts
 - Self-supervised learning: the model learns to predict tokens in existing texts
 - The two branches predict differently:
 - Generative (decoder): predict the *next* token from the previous ones
 - Encoder: predict a hidden (“masked”) token from the tokens around it
 - Generates (most of) model's **weights** (or **parameters**)

Aside: the model is a huge **neural network**, and training means estimating its weights (the algorithms are beyond our scope)

Training

Pretraining creates a model that can 'speak' very realistically

But, it doesn't know how to follow instructions, nor does it have a sense of what it should and shouldn't do

- It's a cute party trick: create text that mimics human speech
- Point of these models: be useful and responsive to humans

2. Posttraining:

- Supervised learning: train on prompt-response pairs (written by humans)
- Reinforcement learning: reinforce good behaviour based on human or programmatic feedback
- Updates model weights to encourage desirable behaviour, known as **alignment**

Posttraining is a generative-branch concern: encoder models aren't conversational; they are instead **fine-tuned** on labelled data for specific tasks

Fine tuning

Many of the most famous LLMs are **general-purpose models**

- This just means they are trained for “general” use
- Consumer facing products like ChatGPT and Claude use general-purpose models

But specialised use cases might require more specialised models that generate text with different properties

Some LLM providers allow users to **fine-tune** their general-purpose models for specific applications

- This entails further training with specialised prompts and responses for your use case
- E.g.: a company building an AI nurse would want it to speak in certain ways that might differ from general conversation

Open versus closed models

One major difference between models: are models **open weight** or **closed weight**?

Open weight models allow users to see the weights generated during the training process

- Can easily fine-tune them
- Examples: Llama, Mistral, DeepSeek

Closed weight models do not expose the weights

- Can only customise within the bounds of tools provided by the companies, such as APIs (and also have to pay)
- Examples: GPT-5, Claude, Gemini

Open versus closed models

There are trade-offs

Closed-weight models:

- Typically perform better (for generic tasks)
- Require less technical expertise (APIs are quite easy to use)

Open-weight models:

- Very customisable
- Can generate more reproducible results
- Ensure data protection (if all inference is on machine)

Model size and “personalities”

Another way models vary is **model size** (number of weights)

- More weights correlate with better performance
- Take more space, memory, computational resources

Each LLM has been post-trained in a specific way, which leads to perceived differences:

- OpenAI's models: more informational
- Anthropic's models: more chatty, emotional and whimsical
- Google's models: more technical

This matters (somewhat) less for data science pipelines than technical details like model size, context windows, etc.

Prompt engineering

What is “prompt engineering”?

Remember: when you use an LLM for inference:

- You're not training the model
 - Caveat: companies *might* use your conversations to “improve their products”
- You're using a (very good) probabilistic text generator

Your prompts are a form of probabilistic conditioning:

- A prompt defines “starting context” for LLM's predictions

Prompt engineering is the practice of carefully constructing prompts to generate useful responses (for task at hand)

What is “prompt engineering”?

Why is this important?

Garbage in, garbage out (GIGO) principle applies: if your prompts are bad, the output will be bad too

But special problem with language: it's full of ambiguities

LLMs have been *trained* to write responses humans like

→ Humans are easily impressed (and persuaded) by articulate language even when *substantive content* is low quality

You must consistently have your guard up against sycophancy and being persuaded by articulate speech

→ E.g.: “You’re right, I apologize”, whether or not you were actually right

Write good prompts and scrutinise the quality of LLM outputs!

Types of prompts

There are two main categories of prompts

1. **Descriptive prompts**

- Example: “Data scientists analyse data and...”
- Model will continue text generically

2. **Instructional prompts**

- Example: “Write a two-sentence summary of what data scientists do.”
- Model follows the command, returns a summary

Instructional prompts are the bread-and-butter of data science applications

Types of instructions

Can get better responses with more *complex* prompting

Chain-of-thought (CoT) prompting

- Instruct the LLM to explain its reasoning path
- Example: “Question: What is 12% of 150? Instruction: Think step by step before answering.”

Reflexive (or meta) prompting

- Instruct the LLM to evaluate and/or revise its own response before finalising it
- Example: “Draft your answer, then check it for logical consistency before giving the final version.”

Types of instructions

Reasoning models are *trained* to do CoT and be reflexive

- They do not “reason” like a human, but they do try to find a better answer

They work by:

- Generating intermediate reasoning steps
- Feeding those steps back into their own context
- Refining their thinking across multiple passes
- Producing a final answer for the user

Reasoning uses more tokens; some models let you see the reasoning text, others hide or summarise it

Types of instructions

Structured prompting

- Instruct LLM to provide machine-readable or consistent output formats
- Example: "Summarise this article. Return JSON with this structure: {\"summary\": \"...\", \"keywords\": [\"...\"]}"

Iterative prompting

- Instruct refining with feedback loops (human or code).

Instruction-following behaviour

Message roles structure how instructions are followed

For many models:

- **system** (or **developer**): messages that set overall behaviour, tone, boundaries, or domain context
 - System messages are applied to entire conversation
- **user**: messages that give the task-specific instruction
 - This is the “prompt” you think of in normal ChatGPT use
- **assistant**: LLM’s response, which is conditioned on both higher-level and user-level instructions

Conditioning versus training

Keep in mind:

- Instructions do not “train” the LLM
- They simply narrow the context for the LLM
- After you end your chat/session, the instructions are lost
- Caveat: stored memories

You can also condition by providing examples

- **Zero-shot** prompting: give LLM a task without examples
- **Few-shot** prompting: give LLM a task with some examples

Zero-shot prompting is common and tempting

- Save money and time
- But: responses will be based on general knowledge

Few-shot prompting

```
prompt <- list()
prompt[[1]] <- list(role = "system",
                    content = "You are an assistant that classifies sentiment.
                               Respond with only the sentiment label.")
prompt[[2]] <- list(role = "user", content = "Text: 'I love this product!'")
prompt[[3]] <- list(role = "assistant", content = "Positive")
prompt[[4]] <- list(role = "user", content = "Text: 'This is the worst service.'")
prompt[[5]] <- list(role = "assistant", content = "Negative")
prompt[[6]] <- list(role = "user",
                    content = "Text: 'The food was okay but the service was slo

response <- chat(model = my.model, messages = prompt, output = "text")
cat(response)
```

Neutral

Prompting for data science pipelines

These techniques matter more for interactive chat than for data pipelines

For data science applications, focus on:

- Clear, templated instructions
- Structured output formats (e.g., request JSON responses)
- Few-shot examples when tasks are ambiguous

Keep prompts simple and consistent when processing lots of data

- Fancy prompting techniques add cost and complexity
- Test on sample data first to ensure outputs are usable

LLMs as a data-science tool

The single most useful LLM task for data science: turn **unstructured text into structured data**.

- Classify documents, extract fields, code open-ended responses
- Exactly the labelling that used to need armies of research assistants

The recipe: loop an LLM over many texts with a **consistent, templated prompt**, and collect the results into a table.

- Recall from yesterday: this is the same goal as the DFM, text → a tidy dataset
- The LLM just does the “reading” for you
- The difference: the DFM got its table by throwing away order and context
- The LLM reads the raw text: trip 2's information loss is no longer the price of a table

A classification pipeline

Suppose we have a batch of customer reviews and want a sentiment label for each.

```
reviews <- c("The food was cold and the staff were rude.",  
            "Absolutely wonderful --- I'll be back!",  
            "It was fine, nothing special.")  
  
classify <- function(text) {  
  p <- list(  
    list(role = "system",  
         content = "Classify the sentiment. Reply with only:  
                    Positive, Negative or Neutral."),  
    list(role = "user", content = text)  
  )  
  chat(model = my.model, messages = p, output = "text")  
}  
  
labels <- vapply(reviews, classify, character(1))
```

A classification pipeline

```
dat <- tibble(review = reviews, sentiment = str_trim(labels))
dat
```

```
# A tibble: 3 x 2
```

review	sentiment
<chr>	<chr>
1 The food was cold and the staff were rude.	Negative
2 Absolutely wonderful --- I'll be back!	Positive
3 It was fine, nothing special.	Neutral

From here it's just data: `count(dat, sentiment)`, plot it, model it.

But are the labels right?

The pipeline runs and produces a clean table, with a label for every review

Three problems:

- The model is **equally confident** when it is wrong
- You have no **ground truth**: did “It was fine” really mean *neutral*?
- At 100,000 reviews, you cannot eyeball the errors

This is exactly where people get into trouble

Validate like any classifier

The LLM is acting as a classifier, so evaluate it like one (Lecture 7)

The standard fix: a **gold-standard sample**

- Hand-label a random sample of the texts yourself
- Compare your labels to the LLM's: accuracy, precision, recall
- Report the agreement alongside your results

If agreement is poor: revise the prompt, add few-shot examples, then validate *again*

Scale changes nothing: 100,000 reviews still only need a few hundred hand-labelled ones

- You cannot skip the reading entirely, you can only *sample* it

Getting more from LLMs

LLMs do not know everything

LLMs are very powerful, but they do not have all the relevant information a user may need

- They know what they were trained on
- What was the corpus used to train them?
- What information was used for alignment?

Many applications require access to specialised information:

- LLMs designed for legal applications need to “know” the law
- LLMs designed for medicine need to “know” medical information
- LLMs designed for customer service (chatbots) need to “know” internal information about the company’s processes

LLMs do not know everything

Lack of knowledge is a problem, but LLMs make it worse

They often **hallucinate** when they don't have info they need

This is a major problem LLM providers are trying to fix

- Models are now more willing to say “I don't know”
- Most models now search the internet for information

Of course, this depends on the model and the question

- GPT models *routinely* give me incorrect code (after zero-shot prompts)

The insidious kind of hallucination

Made-up citations get all the press

- These are the *tame* hallucinations: obvious, easy to catch
- Little damage done (beyond your reputation)

The insidious kind: plausible but wrong

- “This function drops missing values by default” (it doesn’t)
- “It’s standard practice to drop these observations” (it isn’t)
- A fluent summary of a paper that reverses its actual finding
- “Report that amount on line 16 of the form” (it never read the form: it answered from “memory”)

The model isn’t “lying”: it is completing a pattern

- Plausible-sounding output is *exactly what next-token prediction produces*
- “Plausible” and “true” are different properties
- Defence: ask for sources, check them, verify yourself

Retrieval-augmented generation

Retrieval-augmented generation (or **RAG**): *augment* LLM text generation by providing access to database of relevant information

Basic process:

1. Store relevant text chunks in an external database along with their VRs
2. Convert the user's prompt into a VR (here: one vector for the *whole* prompt, not one per token)
3. Retrieve the most similar text chunks from the database using vector similarity search
4. Insert the retrieved text into the prompt
5. Run LLM inference

Retrieval-augmented generation

An example of a medical use: “What are the current contraindications for prescribing Drug X to pregnant patients?”

→ LLM may provide outdated guidance, generic info or hallucinate

With RAG:

- Question is embedded
- System searches a vector database populated with recent clinical guidelines, FDA updates, latest peer-reviewed studies, internal protocols, etc.
- Retrieves relevant sections
- Text of sections are inserted into prompt along with instruction to *only* use provided information
- LLM summarises only authoritative material retrieved from database

Retrieval-augmented generation

Key innovation of RAG:

- Only relevant info from the database is extracted
- You are *not* feeding the entire database into each prompt
- This keeps prompt within the context window

Sounds complicated, but it's fairly easy to do

Closed-weight models have tools, e.g. OpenAI's "file search":

- Upload file(s) to the API
- Include file search tool in your API call, along with file ID
- The server does some sort of RAG (black-boxed)

Beyond text generation: agentic tools

Agentic tools put an LLM in a loop with your computer

The loop (one prompt from you, many calls to the model):

- The model can only write text: to “act”, it writes a *request* (run this command, edit that file)
- The harness executes the request and pastes the result into the context
- The model reads the result and writes the next request
- Repeat until it decides the task is done

The model still only predicts tokens: the harness supplies **context** and **execution**

For data science: multi-step workflows (clean, analyse, report); you will use these daily

- But the loop can also run `rm -rf` on your project(!) – permissions matter

Security and data ethics

Before you use LLMs: data privacy

Your prompts are not private, unless inference is local

- When using APIs or web apps (any frontier model), your prompts are sent to remote servers
- Providers may store, log, or use your data to train future models
- Read and understand the provider's data policy
- **Never** send sensitive data (personal info, confidential research data, proprietary code) unless you know exactly what you're doing

A hosted model is a **third party** to your data agreements

- Working with human-subjects data? Check with your ethics board
- Also: results can be model-dependent, which is bad for reproducibility

For very sensitive data, run local models only

Before you use LLMs: agentic tool risks

Agentic tools can act on your computer

- Tools like Claude Code, Copilot, Codex can read files, run commands, edit code
- A mistake or misunderstanding can delete files, expose secrets, or break systems
- Use sandboxing; review actions before approving

Sensible practices:

- Never paste API keys or passwords into prompts, or leave them in the project folder
- Start narrow, expand deliberately: the agent does not need your whole hard drive
- Don't use the default settings: learn the permission system first

Working with LLMs

The real price: distance

These tools are genuinely amazing: you will use them constantly

These tools produce enormous output for almost no effort

But every piece of work you didn't do yourself is work you are further from

- You didn't write it, so you have to *review* it
- You didn't practise it, so you didn't *learn* it
- You didn't think it through, so you can't *defend* it

Distance is the real cost of these tools; it comes in three forms

Distance 1: more output than you can review

Agents produce code and text faster than you can read carefully

- More output is only better if it is *good*, and the only way to know is to read it

Common problems in LLM-generated code:

- Code that runs, but the logic is wrong
- Hard-coded values; “cheat routes” after repeated failures
- And the errors are the *insidious* kind: plausible, fluent, invisible

Defence: spot checks, review diffs, version control, and keep the volume down

Distance 2: skills you never build

You stop practising the skills the agent performs for you

→ **Cognitive debt**: skill atrophy from non-practice

In school, this is fatal: producing is not learning

- An elegant LLM summary or study guide gives you the **illusion of knowledge**
- Reading it is not the same as being able to *do* it
- Universities are already responding: device-free, pen-and-paper exams
- And we see it in classrooms: students who lean on LLMs struggle with core skills

You cannot delegate a skill you do not yet have

Distance 2: skills you never build

For a data-science career, the stakes are higher still

- These tools *multiply* an analytical foundation; they do not replace one
- A rando can hand their data to an LLM company and vibecode an app for themselves: fine, their data, their app
- You will build products for, and do research on, *other humans*
- They deserve better than the rando: you have to **be the expert**

Think of a pilot:

- Autopilot can fly the plane, but the pilot must still understand what it is doing
- A pilot who does not is dangerous in any cockpit

Distance 3: your name is still on it

Accountability does not transfer to the tool

- You cannot blame the LLM to an examiner, employer, journal, or court
- Undisclosed LLM use on assignments is probably cheating
- People *will* be suspicious of heavy reliance: be ready to disclose and defend your use

The pilot again: the passengers' safety is still the pilot's responsibility

Use these tools to do *better* work, not just *more*

You've learned to fly this term. What remains: the principles of flight.

The limits of our tools

Information is not understanding

A recurring hope: the new tool will finally unlock understanding

- 1990s: regression on every desktop
- 2010s: machine learning on every laptop
- 2020s: LLMs, reading everything and talking back

Every tool on this list helps you learn: each reveals more information than the last

- But information is not understanding

LLMs make the confusion easiest of all

- Persuasive, sycophantic, human-sounding: fluent output *feels* like understanding
- Endless plausible output, and plausible is not true

Data does not explain itself.

Data does not explain itself

At some level, you already know this

Each part of this course taught you a caution:

- Statistics: be careful learning from *samples*
- Causal inference: be careful concluding from *relationships*
- Machine learning: be careful extrapolating from *highly predictive models*

One common thread: be careful drawing inferences from data

But the cautions miss a deeper problem. Let me show you

A social-media argument about the Tube

An angry commuter claims there's far more disorder on the Tube.

The politician in charge of TfL disagrees: two recent rider-satisfaction surveys show satisfaction went *up*.

The data:

- Two simple random samples of 300 transit riders each
- "Satisfied or not?" coded $Y_i = 1$ or 0
- Sample means: $\bar{Y}_1 = 0.680$, $\bar{Y}_2 = 0.737$

Satisfaction went up: does that settle it?

Is the change even real?

Stack the data ($n = 600$) with wave2 (= 1 for the second survey) and regress:

$$Y_i = \beta_0 + \beta_1 \text{wave2}_i + \varepsilon_i$$

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.680	0.026	25.92	<0.001
wave2	0.057	0.037	1.53	0.127

- $\hat{\beta}_1 = 0.057$, $p = 0.127$: *not* statistically significant
- The “increase” isn’t even confirmed by the data

Even if it were, what would it mean?

Suppose the increase *were* real. Would it mean the Tube got better?

- Some riders are bothered by disorder more than others
- If disorder rises, *those* riders stop riding: they select *out* of the second survey
- The riders who remain look more satisfied, even if nothing actually improved

A selection problem: the more-dissatisfied people are missing from the sample.

The data can't tell you which story is true: *you* have to reason about it.

Prediction won't save you either

Set inference aside.

The politician's data scientist fits an ML model to the second survey: predict satisfaction from age, fares, journey history

- Suppose it predicts brilliantly
- It predicts satisfaction *among people still riding*
- The dissatisfied have left: no model can predict the missing
- And feature importance is not an explanation: it tells you *who*, not *why*

A model can predict the sample perfectly and still explain nothing.

Sharp empirical skills are not enough

Every caution in this course protects your inferences.

But none of them produces an explanation:

- Causal inference can show something works, but not why
- Machine learning can predict, but can't give reasons
- An LLM is the dangerous one: it *will* hand you an explanation
 - Easy explanations it knows; hard ones it fakes: the most plausible-sounding story in its training data
 - And the hard cases are precisely the ones you're paid to get right

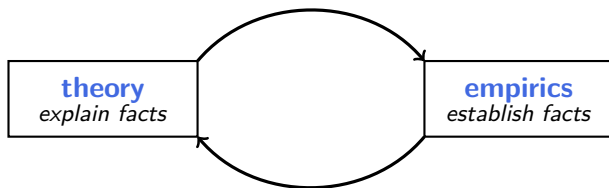
**Sharp empirical skills are not
enough to learn about the world.**

Theory: claims about *why*

Theory is a set of claims about *why* the world is the way it is.

- Not a description of findings, but an explanation for them
- Theory is about the “why,” not the “what”

Learning is a conversation between the two:



Models simplify a complicated world

The world is too complex to grasp all at once. So we need:

a principled way to simplify complexity into something we can reason about.

A **theoretical model** is a tool for making sense of a complicated problem.

- By necessity, models *simplify*
- Good models focus attention on the most important parts
- You use them constantly: an exam-revision outline, an explanation for an election result, a diet plan

You have seen the word “model” a lot in this course, often in the context of empirical observation/inference

- Here: we'll be more abstract, hence *theoretical* model.

Models are like maps



A map of the London Tube.

It is *not* realistic: not to scale, geography distorted.

But it is extremely *useful* to a passenger.

A “realistic” street map (every detail, to scale) is often *less* useful for actually getting around.

Evaluating a model

Facts about maps (and models):

1. They are objects: neither “true” nor “false”
2. They have limited accuracy
3. They are *partial*: they leave things out on purpose

So don't ask “is this model accurate / complete / true?”

Ask instead: **is this model useful for our purpose?**

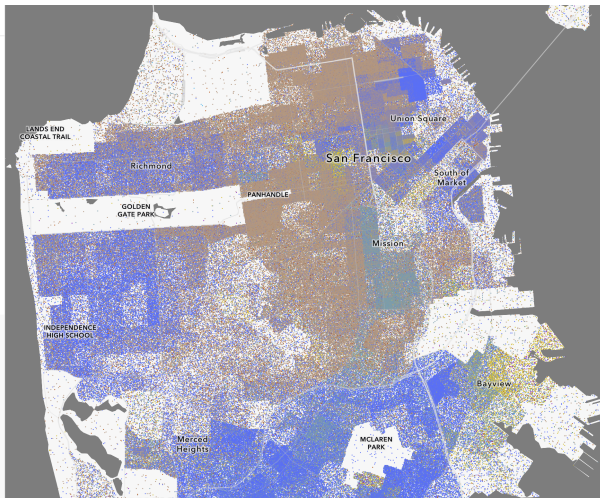
Schelling's model of segregation

Legend

USA Census 2020 Redistricting Block Groups

1 Dot = 2 people

- Non-Hispanic or Latino Population:
American Indian and Alaska Native alone
- Non-Hispanic or Latino Population: Asian
alone
- Non-Hispanic or Latino Population: Black
or African American alone
- Hispanic or Latino Population
- Non-Hispanic or Latino Population: Native
Hawaiian and Other Pacific Islander alone
- Non-Hispanic or Latino Population: Some
Other Race alone
- Non-Hispanic or Latino Population:
Population of two or more races:
- Non-Hispanic or Latino Population: White
alone



Segregation persists. Is discrimination the *only* explanation?

Schelling's model of segregation

A model of a society with two types of people, red and blue:

- Each group is half the population
- Everyone cares *only* about their immediate neighbourhood
- A person is **happy** if at least half their neighbours are like them; otherwise they move to an empty spot

That's it. Notice: nobody wants segregation; they're perfectly happy in a 50/50 neighbourhood.

Try it: https://courses.cs.duke.edu/compsci308/spring20/assign/02_simulation/nifty/mccown-schelling-model-segregation/

What Schelling teaches

Run it, and the society *segregates anyway*.

The insight:

- Even with *pro-integration* preferences, people still sort into segregated communities
- So discrimination and racist attitudes are *not necessary* for segregation (though they're certainly sufficient)

Why this is useful:

- It explains why segregation is so hard to undo
- It warns policymakers: bans alone may not be *enough*
- The simplistic assumptions don't stop us learning: they're what *make* the lesson possible

Partial equilibrium thinking

Schelling's people respond only to their own neighbourhood

Real people also react to *each other*

Partial equilibrium thinking (PET): explaining a pattern while forgetting that people react to the forces you're studying

- Intuitions are often wrong in strategic settings
- The Tube example was PET: riders reacted to disorder by leaving the sample

Next: PET derailing a real research literature (my own field)

Case study: politics in the US courts

Are judges neutral arbiters, or “politicians in black robes”?

The US setting:

- Legal disputes go to trial courts, which issue *judgments*; the national ones are the federal district courts
- District judges are appointed by presidents, so appointments are political

Scholars ask: does a judge's ideology affect how they decide cases?

- Decades of studies estimate this “effect of ideology” using judges' decisions
- Typical finding: little or no effect (so politics only matters at the Supreme Court)

Two mistakes hiding in that design, you now have tools to spot both

Mistake 1: “the effect of ideology”

From causal inference: what is the unit of observation, and what treatment is assigned to it?

“Effect of ideology” answers: the judge, treated with ideology

- No counterfactual: you cannot assign an ideology to a judge
- Scholars proxy ideology with the appointing president’s party, but party is a **compound treatment**: it bundles race, gender, age, training

The world answers differently: clerks randomly assign judges to cases

- The unit is the case; the treatment is the judge it draws
- Assignment to a Republican vs Democratic appointee: well-defined and random
- We care what happens to cases, not what happens inside judges

Random assignment is a stroke of luck: a natural experiment (verify, don’t assume)

Mistake 2: dropping settled cases

Most cases never reach a decision: roughly a third settle, many more are dismissed

- Prior studies kept only judgments, opinions, appeals
- Settled and dismissed cases were dropped

Assignment is randomised at filing; settlement happens after the judge is known

- Subsetting on how a case ended: conditioning on a **post-treatment** variable
- The sample no longer reflects the randomisation!
- Hübert and Copus (2022): dropping these cases biases the effect towards zero

The “null effects” were an artefact of bad research design

What politics actually does

Hübert and Copus (2022): all ~70,000 civil rights cases in seven district courts, 1995–2016

Assignment to a Republican (vs Democratic) appointee causes:

- Fewer settlements and more dismissals
- Pro-defendant outcomes up about 5 percentage points
- No detectable effect on formal judgments

The mechanism: litigants respond to the judge they draw

- Defendants expect to win: fight rather than settle
- Plaintiffs expect to lose: withdraw, or accept less
- Old studies looked where judges act; politics operates where litigants react

PET hid this for decades

Learn to think strategically

Game theory: the toolkit for reasoning about strategic responses

- Schelling, litigants anticipating judges, riders leaving the Tube: all game theory
- It brings the rigour you now demand of estimates to “what would people do if...?”

My advice: take a game theory course

- Most social science and economics programmes offer one
- This course sharpened your empirical thinking; game theory sharpens the other half

Bringing it together

The best data scientists combine two kinds of rigour:

- **Empirical rigour**: the tools from this term (regression, causal inference, machine learning, text analysis, and yes, LLMs)
- **Theoretical rigour**: asking *why* your results look the way they do, and whether your interpretation is logically sound

Neither is enough alone:

- Good theory without data is speculation
- Good data without theory is description

The conversation between them is where understanding happens.

Going forward

- When you find a pattern, resist explaining it immediately: ask what *else* could produce it
- When someone offers an explanation, ask whether the conclusion *actually follows*
- Be suspicious of your own intuitions, especially where people react strategically to the same forces you're studying

You now have the tools to establish facts about the world.

Use them carefully, and pair them with clear thinking about what those facts mean.